A High-Performance Document Image Binarization Pipeline with Parallel Image Processing

Aghiad Khertabeel
aghiad.khertabeel@uni-jena.de
Friedrich Schiller University Jena
Jena, Germany

Mohamad Khaled Minawe
minawe.mohamad.khaled@uni-jena.de
Friedrich Schiller University Jena
Jena, Germany

Abstract

This is a placeholder abstract for the paper.

Keywords

document image binarization, image processing, OCR preprocessing, adaptive thresholding

1 Introduction

The digitization of documents has become increasingly prevalent in various domains, including libraries, archives and businesses. The goal is to revitalize the information contained in the physical documents and make it more accessible and searchable in digital form as well as preserving it for future generations. This process involves scanning physical documents in order to create a digital representation of them. However, the quality of the scanned images can vary significantly due to factors such as lighting conditions, document physical state and the scanning equipment used. As a result, the scanned images may contain noise, shadows, and other artifacts that can hinder the process of the recognition using OCR (Optical Character Recognition) systems [8].

To address such challenges, preprocessing techniques are often applied beforehand in order to enhance the quality of the scanned images and improve the accuracy of OCR systems [5]. One of the most common preprocessing techniques is image binarization, which converts a colorful or a greyscale image into a binary image consisting solely of black and white pixels [10]. According to Gupta et al., this process allows OCR systems to focus on the relevant information in the document and ignore the background, making the OCR systems more accurate [5].

While there are many algorithms for document or image binarization available, such as Niblack's approach and Sauvola's method, these algorithms can be computationally intensive as they often require processing each pixel or each cluster of pixels individually, which can be time-consuming, especially when working with large documents with high DPI [2, 3]. Therefore, there is a growing need for high-performance solutions that entail the usage of parallel processing techniques either on the CPU level or the GPU level. Furthermore, implementing parallelization on the CPU level can be achieved by splitting the work among many worker threads, whereas with GPU-level parallelization, the work can be distributed among many GPU threads, which are usually more efficient than CPU threads when it comes generally to image processing and rendering tasks [1]. This statement was shown by Ansari et al. as they did a performance comparison between parallelized CPU and GPU performance for large metrics and the GPU performance was consistently faster than the CPU by many folds. The reason for that is because GPUs tend to have significantly more cores than CPUs, allowing them to handle a larger amount of smaller computations in parallel.

2 Background

As discussed earlier in the introduction, scanned documents or images often require preprocessing steps to enhance their quality for OCR using binarization techniques. However, binarization in itself could also suffer if the scanned document contain significant noise and therefore the binarization stage may fail to separate the foreground from the background adequately. For that reason, preprocessing techniques can also be applied before the binarization step in order to prevent the noise from affecting the foreground-background separation. Furthermore, post-processing techniques can also be applied to remove artifacts or noise that may be left after the thresholding step or remove dark borders that are usually left over by scanners when documents or books are not placed evenly on the scanner bed.

The literature provides a wide variety of techniques for preprocessing and post-processing for scanned document enhancement, alongside a numerous variety of binarization algorithms. These approaches are incorporated into the processing pipeline proposed in this paper, which is describes in sections 3 and 4.

2.1 Preprocessing

2.1.1 Non-linear Filtering. The most trivial approach for the median filter was discussed by Gonzalez and Woods in their book "Digital Image Processing" [4]. Their algorithm is non-linear filtering approach that operates on each pixel and its surrounding neighborhood to remove salt-and-pepper noise sparsely distributed across the image. The median intensity of the nine pixels in the window is computed (the current pixel and its 8 neighbors) and replaces the current pixel, which is the center pixel, with the calculated median value. Therefore, this filter is effective when the noise is sparse and does not cover large regions of the image, because a dark pixel surrounded by brighter neighboring pixels will yield a larger median intensity value, and thus the median value becomes higher, replacing the corrupted pixel with a brighter intensity value. The reverse also applies when the pixel is a light pixel

surrounded by dark pixels. There are several variations of this filter implementation that are asymptotically more optimized. Common optimizations are proposed by Huang et al. [6] and Perreault et al. [7].

2.1.2 Linear Filtering. A similar approach to the median filter is the background estimation method, which is dicussed by Gonzalez and Woods in their book "Digital Image Processing" [4]. In contrast to median filtering, this approach is linear and operates also on neighboring pixels by computing the sum of the pixel values within a specified window and then dividing by the number of pixels in that window. This method is computationally more efficient than the median filter, because to compute a median value, the values requires either sorting or a selection algorithm such as *median of medians* has to be used. Although *median of medians* asymptotically also has linear asymptotic time complexity, it still requires more computations per iteration compared to simple accumulation of pixel values and summing them up.

2.1.3 Contrast Stretching. Some image documents may suffer from low contrast and therefore have a narrow range of pixel intensities between foreground and background pixels. In such cases, improving the contrast between the foreground and background can help the binarization process as well as the OCR later on. The main idea is to make darker pixels more dark and lighter pixels more light, so that the contrast in the image improves overall. There are several approaches for contrast stretching, the simplest of which is the global linear contrast stretching method, which is discussed by Gonzalez and Woods [4]. Other optimizations were also proposed in the literature, such as the contribution from Su et al. [9], where they proposed a local contrast stretching strategy instead of global-based stretching. The benefit of their approach lies in the fact that their method is more robust to local variations in the image with uneven illumination and degradation. Compared to the global approaches, the local approach is less sensitive to max/min outliers in the image. This is usually the case when the image is heavily degraded.

2.2 Processing

2.3 Post-processing

3 System Architecture

The system implements a Scanned Document Enhancer designed for document quality improvement before printing or OCR processing. The architecture follows a modular C++ pipeline design with clear separation of concerns, enabling maintainable and testable code.

The processing utilizes sequential pipeline stages, allowing users to customize the enhancement workflow based on their specific needs.

OpenMP is also used for parallelization throughout compute-intensive operations to leverage multi-core CPU capabilities, significantly reducing processing time for large documents. The system uses the PPM P3 ASCII format for input and output images, which simplifies file handling and allows for easy debugging and visualization of intermediate results.

The design follows header/implementation separation, with clear interfaces defined in header files and implementation details encapsulated in source files. This promotes code reusability and maintainability, allowing easy extension of the system with additional enhancement techniques in the future.

3.1 Main Components

The main components can be seen in the directory structure in Listing 1:

The CLI layer, implemented in `main.cpp`, serves as the entry point of the application and the pipeline orchestration module. This layer deals with parsing arguments and configuration management.

The IO layer, located in the `io` directory, is responsible for reading and writing PPM images. It includes modules for handling the PPM P3 ASCII format, such as `ppm_reader` and `ppm_writer`, as well as a `token_reader`. The `ppm_reader` and `ppm_writer` modules handle the specifics of reading and writing PPM files, while the `token_reader` module provides utility functions for parsing the ASCII format.

The Image Processing layer, located in the `image` directory, contains `GrayImage` as the main data structure for representing grayscale images. It contains the width, height, maxval, and flat vector data, along with colorspace utilities for converting between RGB and grayscale color spaces.

The Operations layer, located in the `ops` directory, contains various image enhancement techniques, including denoising (median filter), background estimation and removal, contrast stretching, thresholding methods (Otsu's, Sauvola's, Nick's, Su's, and a proposed method), morphology operations (open and close), and border cleanup. Each operation is implemented in its own module, allowing modularity and ease of maintenance.

Several operation modules utilize integral images for efficient computation of local statistics, while others rely on direct neighborhood processing. Additionally, OpenMP is used for parallelization to ensure high performance even when processing large documents.

The Utility Layer in util directory provides `args_parser` for command-line argument processing, logging for structured output and debugging, timing for performance measurement utilities, and clamp for mathematical helper functions. The Testing Framework in tests/ uses Catch2-based testing with 56 test cases and 1038 assertions, covering component testing for I/O and algorithms as well as integration testing, with validation for algorithm correctness and edge case handling.

3.2 Data Flow

The high-level pipeline follows the sequence:

(1) The user invokes the application via the command line, providing input and output file paths along with optional parameters for the enhancement pipeline.

(2) The CLI layer parses the arguments and configures the processing pipeline accordingly.
(3) The IO layer reads the input PPM image and converts it to a `GrayImage` format for processing.
(4) The Image Processing layer applies the selected enhancement operations sequentially, with each operation taking the output of the previous one as input. Operations may utilize integral images for efficient local computations and OpenMP for parallel processing.
(5) After processing, the final enhanced image is written back to a PPM file by the IO layer.

The detailed processing steps begin with the Input Stage performing PPM P3 file parsing and validation. Color Conversion transforms RGB to grayscale using standard luminance weights. The Preprocessing Chain includes optional median denoising with a 3×3 kernel, background estimation via separable box blur, background removal with target brightness adjustment, and percentile-based contrast stretching.

The Binarization Stage applies one of five configurable methods to the preprocessed image. The Postprocessing Chain performs optional morphological operations using a 3×3 structuring element and border cleanup with either fixed width or adaptive dark pixel removal. The Output Stage writes the final binary image as PPM P3 format.

Memory Management utilizes single-pass processing to minimize memory footprint, in-place operations where algorithmically feasible, integral images precomputed for adaptive thresholding methods, and OpenMP parallelization with thread-local variables and reduction operations. This architecture ensures modularity, testability, and performance while maintaining code clarity and extensibility.

4 implementation

In this section, we will describe the implementation of all mentioned methods in the Processing layer and Operation layer of the system, starting with the denoising method, then background estimation and removal, followed by the binarization method, and finally the morphological operations. We will also discuss the parallelization techniques used in each method and how they contribute to the overall performance of the system.

4.1 Median Filter for Image Denoising

For the denoising method, we implemented a median filter, which is a non-linear digital filtering technique. The median filter works by replacing each pixel value with the median value of the pixels in its 3×3 neighborhood. This method is effective in removing salt-and-pepper noise while preserving image edges.

In our implementation, we iterate over every pixel, collect the values of its neighboring pixels, compute the median using std::nth_element, and assign it to the output image. For border pixels, the coordinates are clamped so that the nearest valid edge pixels are reused.

To improve performance, the filter is parallelized with OpenMP, allowing multiple image rows to be processed simultaneously by different threads.

4.2 Background Estimation

To estimate the background of the image, we implemented a separable box blur filter. The box blur computes the average intensity of pixels within a square neighborhood defined by a radius r. Instead of computing a full two-dimensional convolution, the filter is implemented as two one-dimensional passes: a horizontal pass followed by a vertical pass. This technique significantly reduces computational complexity.

Formally, the horizontal pass computes an intermediate image

$$I_{\text{tmp}}(x, y) = \frac{1}{2r+1} \sum_{i=-r}^{r} I\big(\text{clamp}(x+i), y\big),$$

where $I(x, y)$ is the intensity of the original image at position (x, y), $I_{\text{tmp}}(x, y)$ is the intermediate image produced after the horizontal blur, r is the radius of the blur window, $2r + 1$ is the width of the averaging window, and $\text{clamp}(\cdot)$ restricts the coordinate to valid image boundaries.

The vertical pass computes the final background image as

$$I_{\text{background}}(x, y) = \frac{1}{2r+1} \sum_{j=-r}^{r} I_{\text{tmp}}\big(x, \text{clamp}(y+j)\big).$$

where $I_{\text{background}}(x, y)$ is the estimated background intensity, $I_{\text{tmp}}(x, y)$ is the horizontally blurred intermediate image, r is the blur radius, and $\text{clamp}(\cdot)$ ensures that coordinates remain within valid image bounds.

Equivalently, this corresponds to averaging over a square neighborhood of size $(2r + 1) \times (2r + 1)$:

$$I_{\text{background}}(x, y) = \frac{1}{(2r+1)^2} \sum_{i=-r}^{r} \sum_{j=-r}^{r} I\big(\text{clamp}(x{+}i), \text{clamp}(y{+}j)\big).$$

where (x, y) denotes the pixel position, $I(x, y)$ is the intensity of the original image, r is the blur radius, and $(2r + 1)^2$ represents the total number of pixels in the square neighborhood.

In the horizontal pass, a sliding window of size $(2r + 1)$ moves across each row of the image. The sum of the pixel values inside the window is updated efficiently by subtracting the value leaving the window and adding the value entering the window. The average value of the window is then written to a temporary image.

In the vertical pass, the same sliding-window approach is applied column-wise to the temporary image to produce the final blurred background image.

For border pixels, coordinates are clamped so that the nearest valid edge pixels are reused. To improve performance, both passes are parallelized using OpenMP, allowing multiple rows or columns to be processed simultaneously by different threads.

4.3 Background Removal

After estimating the background, it is removed from the denoised image through pixel-wise subtraction. For each pixel, the background intensity is subtracted from the denoised image intensity, and a constant target value is added to maintain a suitable brightness level.

Formally, the output pixel value is computed as:

$$I_{\text{out}}(x, y) = \text{clamp}\big(I_{\text{denoised}}(x, y) - I_{\text{background}}(x, y) + T\big),$$

where $I_{\text{denoised}}(x, y)$ is the intensity of the denoised image, $I_{\text{background}}(x, y)$ is the estimated background intensity T is a constant target intensity added to maintain brightness, $I_{\text{out}}(x, y)$ is the final output image after background removal, and $\text{clamp}(\cdot)$ restricts the result to the valid intensity range.

This operation highlights foreground structures by removing slowly varying background illumination while ensuring that pixel values remain within the allowed intensity range.

4.4 Contrast Stretching

Contrast stretching is an image enhancement technique used to improve the contrast of a grayscale image by expanding the range of intensity values. In many images, the pixel intensities occupy only a small portion of the available dynamic range, which results in a low-contrast appearance. Contrast stretching addresses this problem by applying a linear transformation that maps the minimum and maximum intensity values of the image to the full available range.

In our implementation, we first determine the minimum and maximum pixel intensity values present in the image, denoted by I_{min} and I_{max}. Each pixel value I is then transformed using the following linear scaling function:

$$I' = \frac{I - I_{\text{min}}}{I_{\text{max}} - I_{\text{min}}} \times I_{\text{maxval}}$$

where I is the original pixel intensity, I' is the transformed pixel intensity, I_{min} is the minimum intensity value in the image, I_{max} is the maximum intensity value in the image, and I_{maxval} is the maximum possible intensity value (e.g., 255 for an 8-bit image).

This transformation linearly stretches the intensity values so that the smallest value becomes 0 and the largest value becomes I_{maxval}. As a result, the full dynamic range of the image is utilized, which increases the visual contrast.

In addition to the standard contrast stretching method, we also implement a percentile-based variant. Instead of using the absolute minimum and maximum intensity values, lower and upper thresholds are determined using selected percentiles of the image histogram. This approach reduces the influence of outlier pixels and produces a more robust contrast enhancement.

4.5 Binarization methods

The binarization phase is responsible for converting the preprocessed grayscale image into a binary image consisting of only black and white pixels. In our implementation, we provide several methods for binarization, including Otsu's method, Sauvola's method, Nick's method, Su's method, and a proposed method. Most of these methods are implemented as described in the paper, but for some of these methods, such as the proposed method and the Su, Lu, and Tu method, we applied some changes to the original method in order to make it more efficient for some use cases. These modifications, alongside the implementation, will be discussed in the next sections.

4.5.1 Otsu's Method. In our implementation, Otsu's method is used as a global thresholding technique to convert the grayscale image into a binary image. The algorithm first computes the histogram of the image intensities. Each histogram entry counts the number of pixels having a specific intensity value.

Using this histogram, the algorithm iterates over all possible threshold values t in the intensity range $[0, I_{\text{max}}]$. For each candidate threshold, the pixels are divided into two classes: background and foreground.

The number of pixels belonging to each class is computed as:

$$w_B(t) = \sum_{i=0}^{t} h(i), \qquad w_F(t) = N - w_B(t),$$

where $h(i)$ is the histogram value at intensity level i, $w_B(t)$ is the number of pixels in the background class, $w_F(t)$ is the number of pixels in the foreground class, and N is the total number of pixels in the image.

The mean intensity of each class is then calculated as:

$$\mu_B(t) = \frac{\sum_{i=0}^{t} i \cdot h(i)}{w_B(t)}, \qquad \mu_F(t) = \frac{\sum_{i=t+1}^{I_{\text{max}}} i \cdot h(i)}{w_F(t)}.$$

Using these values, the between-class variance is computed as:

$$\sigma_B^2(t) = w_B(t)\, w_F(t)\, (\mu_B(t) - \mu_F(t))^2.$$

The algorithm selects the threshold t^* that maximizes the between-class variance:

$$t^* = \arg\max_t \sigma_B^2(t).$$

After the optimal threshold is found, the binarization step is applied pixel-wise to the image. Each pixel is assigned either black or white depending on its intensity relative to the threshold:

$$I_{\text{out}}(x, y) = \begin{cases} 0, & I(x, y) \le t^* \\ I_{\text{max}}, & I(x, y) > t^* \end{cases}$$

where $I(x, y)$ is the original pixel intensity, $I_{\text{out}}(x, y)$ is the resulting binary pixel value, t^* is the optimal threshold computed by Otsu's method, and $I_{\max}$ is the maximum intensity value (e.g., 255).

In the implementation, the final binarization step is parallelized using OpenMP, allowing multiple pixels to be processed simultaneously.

4.5.2 Nick's Method. Nick's method is implemented as a local thresholding algorithm that computes a threshold for every pixel based on the intensity statistics of its surrounding neighborhood.

To efficiently compute local statistics for each pixel, two integral images are constructed: one storing the cumulative sum of intensities and another storing the cumulative sum of squared intensities. These integral images allow the sum of pixel values inside any rectangular region to be computed in constant time.

For each pixel (x, y), a square window centered at the pixel with radius r is considered. If the window extends outside the image boundaries, it is clipped so that only valid image pixels are included. The number of pixels inside the window is denoted by $A(x, y)$.

Using the integral images, the sum of intensities $S(x, y)$ and the sum of squared intensities $S_2(x, y)$ inside the window are computed. From these values, the local mean and the second moment are calculated as: :

$$m(x, y) = \frac{S(x, y)}{A(x, y)}, \qquad M_2(x, y) = \frac{S_2(x, y)}{A(x, y)}.$$

where $A(x, y)$ is the number of pixels inside the local window.

The local variance is then obtained using

$$\text{Var}(x, y) = M_2(x, y) - m(x, y)^2.$$

If numerical inaccuracies produce slightly negative variance values, the value is clamped to zero.

The threshold for the current pixel is computed using the Nick formula

$$T(x, y) = m(x, y) + k\sqrt{\text{Var}(x, y) + m(x, y)^2}.$$

The original pixel value $I(x, y)$ is compared with the threshold $T(x, y)$. If the pixel intensity is less than or equal to the threshold, the output pixel is set to 0 (black); otherwise it is set to $I_{\max}$ (white).

The threshold computation and binarization are performed for every pixel in the image. To improve performance, this step is parallelized using OpenMP so that multiple image rows can be processed simultaneously.

4.5.3 Sauvola's Method. Sauvola's method is implemented as a local thresholding algorithm in which a separate threshold is computed for each pixel based on the intensity statistics of its surrounding neighborhood.

To efficiently compute local statistics for every pixel, two integral images are constructed: one storing the cumulative sum of intensities and another storing the cumulative sum of squared intensities. These integral images allow the sum of pixel values inside any rectangular window to be obtained in constant time.

For each pixel (x, y), a square window centered at the pixel with radius r is considered. If the window extends outside the image boundaries, it is clipped so that only valid image pixels are included. The number of pixels inside the window is denoted by $A(x, y)$.

Using the integral images, the sum of intensities $S(x, y)$ and the sum of squared intensities $S_2(x, y)$ inside the window are computed. From these values, the local mean, variance, and standard deviation are calculated as:

$$m(x, y) = \frac{S(x, y)}{A(x, y)}, \qquad M_2(x, y) = \frac{S_2(x, y)}{A(x, y)},$$

$$\text{Var}(x, y) = M_2(x, y) - m(x, y)^2, \qquad \sigma(x, y) = \sqrt{\text{Var}(x, y)}.$$

If numerical inaccuracies produce a slightly negative variance, the value is clamped to zero before computing the square root.

The local threshold is then computed using Sauvola's formula

$$T(x, y) = m(x, y)\left(1 + k\left(\frac{\sigma(x, y)}{R} - 1\right)\right),$$

where k is a user-defined parameter and the constant value $R = 125$ is used in the implementation.

After computing the threshold, the original pixel value $I(x, y)$ is compared with $T(x, y)$. If the pixel intensity is less than or equal to the threshold, the output pixel is set to 0 (black); otherwise it is set to $I_{\max}$ (white).

The threshold computation and binarization are performed for every pixel in the image. To improve performance, this step is parallelized using OpenMP so that multiple image rows can be processed simultaneously.

4.5.4 Proposed Method. The proposed method is implemented as a local thresholding algorithm that combines global image statistics with local window statistics.

To efficiently compute the required local statistics, two integral images are constructed: one storing the cumulative sum of pixel intensities and another storing the cumulative sum of squared intensities. These integral images allow the sum of intensities inside any rectangular window to be obtained in constant time.

Using the integral image of intensities, the global mean intensity m_g of the entire image is computed once at the beginning of the algorithm.

For each pixel (x, y), a square window with radius r centered at the pixel is considered. If the window extends beyond the image borders, its coordinates are clipped so that only valid pixels are included. The number of pixels inside the window is denoted by $A(x, y)$.

Using the integral images, the sum of intensities $S(x, y)$ and the sum of squared intensities $S_2(x, y)$ inside the window are computed. From these values, the local mean and variance are calculated as:

$$m_w = \frac{S(x,y)}{A(x,y)}, \qquad M_2 = \frac{S_2(x,y)}{A(x,y)},$$

$$\mathrm{Var} = M_2 - m_w^2.$$

The local standard deviation of the window is then obtained as:

$$\sigma_w = \sqrt{\mathrm{Var}}.$$

During a first pass over the image, the standard deviation values σ_w are computed for all windows in order to determine the minimum and maximum standard deviation values, $\sigma_{\min}$ and $\sigma_{\max}$.

Using these values, the adaptive standard deviation is computed as:

$$\sigma_{Adaptive} = \frac{\sigma_w - \sigma_{\min}}{\sigma_{\max} - \sigma_{\min}}.$$

The threshold for each pixel is then calculated using the proposed formula

$$T = m_w - \frac{m_w^2 - \sigma_w}{(m_g + \sigma_w)(\sigma_{Adaptive} + \sigma_w)}.$$

Modification of the Proposed Method. In the original formulation presented in the paper, all pixel intensity values are normalized to the range $[0, 1]$ before computing the threshold. In our implementation, the algorithm operates directly on the raw grayscale values in the range $[0, I_{\max}]$. Therefore, the adaptive standard deviation $\sigma_{Adaptive}$, which is originally computed in the normalized range $[0, 1]$, is converted back to the raw intensity scale to maintain unit consistency within the threshold equation.

This conversion is performed as:

$$\sigma_{Adaptive}^{raw} = \sigma_{Adaptive} \cdot \sigma_{\max}.$$

The resulting value is then combined with the local standard deviation σ_w in the denominator of the threshold formula. This modification ensures that all quantities in the threshold computation are expressed in the same intensity scale.

After computing the threshold T, the pixel intensity $I(x, y)$ is compared with the threshold. If the intensity is smaller than T, the output pixel is set to 0 (black); otherwise it is set to $I_{\max}$ (white).

The threshold computation and binarization are performed for every pixel in the image. To improve performance, the processing of image rows is parallelized using OpenMP so that multiple pixels can be processed simultaneously.

4.5.5 Su, Lu, Tan (2010) Method. The method proposed by Su, Lu, and Tan (2010) is designed for the binarization of degraded document images, particularly historical documents affected by uneven illumination, background noise, and ink bleeding. The method first detects high-contrast pixels around text stroke boundaries and then estimates a local threshold based on these pixels.

Contrast Image Construction. The algorithm begins by constructing a contrast image that highlights pixels located around text stroke boundaries. Instead of using the traditional image gradient, the contrast is computed using the local maximum and minimum intensities within a small neighborhood window. For each pixel (x, y), the contrast value is defined as:

$$D(x,y) = \frac{f_{\max}(x,y) - f_{\min}(x,y)}{f_{\max}(x,y) + f_{\min}(x,y) + \epsilon},$$

where $f_{\max}(x, y)$ and $f_{\min}(x, y)$ denote the maximum and minimum intensities inside a 3×3 neighborhood centered at (x, y), and ϵ is a small positive constant used to avoid division by zero. This formulation normalizes the local intensity difference and reduces the effect of illumination variation across the document image. As a result, pixels located near text stroke boundaries typically produce higher contrast values.

High Contrast Pixel Detection. After computing the contrast image, high-contrast pixels are detected using global thresholding. The Otsu method is applied to the contrast image to automatically determine a threshold value t_{Otsu}. Pixels with contrast values greater than or equal to this threshold are classified as high-contrast pixels, which usually correspond to text stroke boundaries.

Formally, a binary edge mask $E(x, y)$ is defined as:

$$E(x,y) = \begin{cases} 1 & \text{if } D(x,y) \geq t_{Otsu} \\ 0 & \text{otherwise.} \end{cases}$$

These detected pixels provide reliable information about the location of text strokes.

Stroke Width Estimation. The size of the local neighborhood window and the minimum number of edge pixels required for classification depend on the approximate width of the text strokes. The stroke width is estimated from the contrast image by analyzing the distances between adjacent peak contrast pixels along each image row.

A histogram of distances between consecutive peak pixels is constructed, and the most frequent distance is taken as an estimate of the text stroke width. This value is then used to determine the window radius r and the minimum number of edge pixels $N_{\min}$.

Local Threshold Estimation. For each pixel (x, y), a square window of radius r is considered. Within this window, the number of detected high-contrast pixels is denoted by N_e. If the number of edge pixels satisfies the condition $N_e \geq N_{\min}$, the local threshold is computed using the intensity statistics of these high-contrast pixels.

Let E_{mean} and E_{std} denote the mean and standard deviation of the intensities of the detected edge pixels within the window. The classification rule is then defined as:

$$R(x,y) = \begin{cases} 1 & \text{if } N_e \geq N_{\min} \text{ and } I(x,y) \leq E_{\text{mean}} + \frac{E_{\text{std}}}{2}, \\ 0 & \text{otherwise.} \end{cases}$$

Pixels satisfying the condition are classified as foreground (text), while the remaining pixels are classified as background.

Modifications in the Implementation. Although the overall structure follows the method described in the original paper, several modifications were introduced in our implementation to improve robustness when processing real-world images.

First, during stroke width estimation, very small peak-to-peak distances are ignored. In the current implementation, distances smaller than 7 pixels are excluded from the histogram, since such distances often originate from noise or background texture rather than actual text strokes.

Second, the estimated stroke width is converted into the local window parameters by forcing the width to be odd. If the estimated stroke width is W_s, then the radius is set to

$$r = \frac{W_s - 1}{2}$$

and the minimum required number of edge pixels is set to

$$N_{\min} = W_s.$$

Third, when the number of detected edge pixels within the local window is smaller than $N_{\min}$, a fallback strategy is used. Instead of relying only on edge pixels, the mean intensity of all pixels in the local window is computed, and the threshold is set to

$$T = \mu_{\text{local}} - 10.$$

This makes the fallback more conservative and reduces the risk of bright background regions being misclassified as foreground. At the same time, it ensures that a reasonable threshold is available even when the number of detected edge pixels within the local window is insufficient.

Finally, integral images are used to efficiently compute both edge-based statistics and the sum of all pixel intensities inside local windows, allowing constant-time evaluation per pixel.

These modifications improve the stability of the algorithm when processing natural photographs and degraded document images while preserving the core principles of the original Su-Lu-Tan method.

4.6 Morphological Operations

Our implementation provides four morphological operations: erosion, dilation, opening, and closing. All operations use a fixed 3×3 neighborhood around each pixel.

Erosion. For each pixel (x, y), the algorithm inspects all pixels in the 3×3 neighborhood centered at that location. The output pixel is set to 0 only if all pixels within the neighborhood are equal to 0. Otherwise, the output pixel is set to $I_{\max}$.

$$I_{\text{erode}}(x, y) = \begin{cases} 0, & \text{if } I(x+i, y+j) = 0 \ \forall (i, j) \in [-1, 1]^2 \\ I_{\max}, & \text{otherwise} \end{cases}$$

Dilation. For dilation, the same 3×3 neighborhood is examined. If at least one pixel within the neighborhood is equal to 0, the output pixel becomes 0. Otherwise, the output pixel is set to $I_{\max}$.

$$I_{\text{dilate}}(x, y) = \begin{cases} 0, & \text{if } \exists (i, j) \in [-1, 1]^2 \text{ such that } I(x+i, y+j) = 0 \\ I_{\max}, & \text{otherwise} \end{cases}$$

Opening. The opening operation is implemented as a sequential combination of erosion followed by dilation:

$$I_{\text{open}} = \text{dilate}(\text{erode}(I)).$$

Closing. The closing operation is implemented as dilation followed by erosion:

$$I_{\text{close}} = \text{erode}(\text{dilate}(I)).$$

The neighborhood indices are clamped to the valid image range in order to handle border pixels. The computations are parallelized using `OpenMP`, allowing different image rows to be processed simultaneously.

5 Conclusion

In this paper, we presented a high-performance document image binarization pipeline that integrates multiple preprocessing techniques and binarization methods, all accelerated through OpenMP parallelization. The pipeline was implemented in a modular C++ architecture, allowing flexible composition of denoising, background estimation and removal, contrast stretching, and five different binarization algorithms: Otsu's method, Nick's method, Sauvola's method, the Su-Lu-Tan method, and a proposed adaptive method.

Table 1 presents the visual results of all implemented binarization methods applied to a sample Arabic calligraphy document. For the preprocessing variant, only contrast stretching with percentile-based thresholds was applied before binarization.

The results show that the choice of binarization method and the use of preprocessing have a significant impact on the output quality. Otsu's method, as a global thresholding technique, produced clean results with well-preserved text strokes in both variants. Since it computes a single global threshold, its output is less sensitive to local contrast variations, making it effective even without preprocessing on this particular image.

Sauvola's method, on the other hand, failed to produce a usable result without preprocessing, yielding a nearly blank output. This is because Sauvola's local threshold depends heavily on the local standard deviation, and the low contrast of the original image caused the threshold to remain too high across most regions. After applying contrast stretch percentile preprocessing, Sauvola's method recovered and produced a result with well-defined strokes, although some background noise artifacts appeared around the text boundaries.

Nick's method with preprocessing produced results comparable to Sauvola's preprocessed output, with clear stroke shapes but

noticeable speckle noise in the background regions. The Su-Lu-Tan method exhibited background noise in both configurations, with the preprocessed variant introducing heavier speckle artifacts throughout the image. This is likely due to the contrast-based edge detection step being affected by the enhanced intensity range after preprocessing. The proposed method is sensitive to the choice of window radius. Without preprocessing and with a large radius, portions of the text strokes appeared but with significant loss of detail. However, with a smaller radius such as 10 (corresponding to a 21×21 window), the text failed to appear entirely. With preprocessing, the proposed method using the appropriate radius recovered most of the stroke detail but introduced considerable background noise.

Overall, the results demonstrate that preprocessing with contrast stretch percentile is essential for local adaptive methods such as Sauvola's and the proposed method when applied to low-contrast document images. Otsu's global approach proved to be the most robust on this sample, while the local methods offer potential advantages on documents with spatially varying illumination or degradation when paired with appropriate preprocessing. The OpenMP parallelization applied throughout the pipeline ensures that all methods can process large document images efficiently on multi-core systems.

References

[1] Mufakir Qamar Ansari and Mudabir Qamar Ansari. Accelerating matrix multiplication: A performance comparison between multi-core cpu and gpu, 2025.

[2] Ankush Mittal Debashish Ghosh Brij Mohan Singh, Rahul Sharma. Parallel implementation of niblackís binarization approach on cuda. *International Journal of Computer Applications*, 32(2):22–27, October 2011.

[3] Xin Chen, Yuefang Gao, and Zhonghong Huang. Cuda-accelerated fast sauvola's method on kepler architecture. *Multimedia Tools Appl.*, 74(24):11809–11820, December 2015.

[4] Rafael C. Gonzalez and Richard E. Woods. *Digital image processing*. Prentice Hall, Upper Saddle River, N.J., 2008.

[5] Maya R. Gupta, Nathaniel P. Jacobson, and Eric K. Garcia. Ocr binarization and image pre-processing for searching historical documents. *Pattern Recognition*, 40(2):389–397, 2007.

[6] T. Huang, G. Yang, and G. Tang. A fast two-dimensional median filtering algorithm. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 27(1):13–18, 1979.

[7] Simon Perreault and Patrick Hebert. Median filtering in constant time. *IEEE Transactions on Image Processing*, 16(9):2389–2394, 2007.

[8] Mohamed Ali Souibgui and Yousri Kessentini. De-gan: A conditional generative adversarial network for document enhancement. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(3):1180–1191, March 2022.

[9] Bolan Su, Shijian Lu, and Chew Lim Tan. Robust document image binarization technique for degraded document images. *IEEE Transactions on Image Processing*, 22(4):1408–1417, 2013.

[10] Chris Tensmeyer and Tony Martinez. Historical document image binarization: A review. *SN Comput. Sci.*, 1(3), May 2020.

A System Architecture Details

Listing 1: Project directory structure

```
project/
|-- CMakeLists.txt
|-- data/
|   |-- in.ppm
|   `-- out.ppm
|-- src/
|   |-- main/
|   |   `-- main.cpp
|   |-- image/
|   |   |-- image_gray.h/.cpp
|   |   `-- colorspace.h/.cpp
|   |-- io/
|   |   |-- ppm_reader.h/.cpp
|   |   |-- ppm_writer.h/.cpp
|   |   `-- token_reader.h/.cpp
|   |-- ops/
|   |   |-- denoise_median.h/.cpp
|   |   |-- background_estimate.h/.cpp
|   |   |-- background_remove.h/.cpp
|   |   |-- contrast_stretch.h/.cpp
|   |   |-- threshold_otsu.h/.cpp
|   |   |-- threshold_sauvola.h/.cpp
|   |   |-- threshold_nick.h/.cpp
|   |   |-- threshold_su.h/.cpp
|   |   |-- threshold_proposed.h/.cpp
|   |   |-- morphology.h/.cpp
|   |   `-- border_cleanup.h/.cpp
|   `-- util/
|       |-- args_parser.h/.cpp
|       |-- logging.h/.cpp
|       |-- timing.h/.cpp
|       `-- clamp.h
`-- tests/
    |-- io/
    |   `-- test_ppm_io.cpp
    `-- ops/
        |-- test_otsu.cpp
        |-- test_sauvola.cpp
        `-- test_median.cpp
```

B Results of The Implemented Binarization Methods

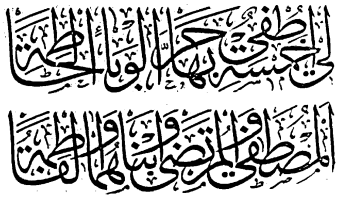
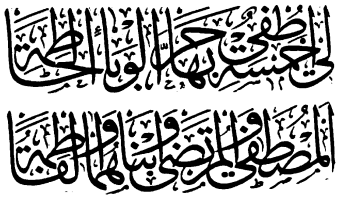
Method	Without Preprocessing	With Preprocessing
Original		
Otsu		
Nick	–	
Sauvola		
Su		
Proposed		

Figure 1: Binarization results of all implemented methods, without and with preprocessing (contrast stretch percentile).